FPGA Lab Final Report

Tyler Wolfe, Jenico Preston, Jose Gonzalez

ECE 3337-301

Dr. Derek Johnston

5/9/2026

Abstract

This project presents the design and implementation of a retro-style gaming console developed on the Basys-3 FPGA platform. The objective of the system was to recreate the architecture of early video game consoles by integrating custom hardware components responsible for processing game logic, generating graphics, and handling user input. Unlike modern gaming systems that rely primarily on software and high-performance processors, this design implements core system functionality directly in digital hardware using Verilog. The console architecture includes a custom central processing unit (CPU), a graphics subsystem capable of generating VGA video output, system memory implemented using FPGA Block RAM, and external hardware components including a game cartridge and player controller. The CPU executes instructions defined by a custom instruction set architecture and interacts with the graphics subsystem through memory-mapped registers. A sprite-based graphics system generates a VGA signal at a resolution of 640 × 480 with a refresh rate of 60 Hz, allowing graphical objects and gameplay elements to be displayed on a monitor in real time. To support software development for the platform, a custom assembler written in Python was developed to translate assembly programs into machine code compatible with the CPU architecture. Additional hardware expansion was implemented through custom printed circuit boards, including a cartridge interface for external 27C512 EPROM-based game storage and a controller PCB for player input through the FPGA PMOD interface. Testing and validation were performed using both simulation and hardware demonstrations. Results confirmed correct operation of the CPU data path, instruction execution, memory access operations, controller interfacing, cartridge communication, and VGA graphics generation. Final system integration demonstrated successful execution of game programs, real-time controller input processing, and stable sprite-based

gameplay rendering on the FPGA platform. These results demonstrate the successful implementation of a fully functional FPGA-based retro gaming console using custom digital hardware and supporting software tools.

Table of Contents

List of Figures

Chapter 1: Introduction and Problem Statement

1.1 Context and Motivation

The goal of this project is to design and implement a fully functional retro styled gaming console capable of running an original video game. The system must be built on the Basys-3 FPGA board and include the hardware and software components necessary to support the gameplay. Unlike modern gaming systems that rely heavily on powerful CPUs and software based graphics, this console requires several core systems to be implemented directly in the hardware. A custom CPU must be designed in Verilog to execute instructions, handle game logic, and manage communication between system components. In addition, a graphics engine must be implemented to generate VGA video output and render objects on the screen itself. The console must also support an interchangeable cartridge that the game ROM itself will be stored on. When inserted into the console, the cartridge provides the program instructions that the CPU will execute, mimicking the architecture used by many earlier gaming consoles. User input will be provided through a custom designed NES style controller with direction and action buttons. The system must continuously read these inputs and update the game state accordingly. This results in a complete embedded system where hardware, peripherals, and software all work together to play a game.

1.2 Engineering Requirements and Specifications

To successfully demonstrate the functionality of the gaming console, the system must meet the technical requirements below. FPGA and logic requirements include:

- A custom CPU implemented in Verilog capable of executing game instructions.

- Support for basic input/output operations and system control.
- A graphics subsystem capable of generating a 640 × 480 VGA signal at 60 Hz.
- Support for rendering sprites and background graphics.
- Hardware logic capable of detecting collisions between game objects.

Memory architecture requirements include:

- Use of FPGA Block RAM (BRAM) for instruction memory and video memory.
- A defined memory map separating program ROM and working RAM.
- Video memory used for storing graphical data used by the display system.

Hardware and peripheral requirements include:

- VGA output using the Basys-3 onboard VGA port.
- Support for at least one external controller connected through PMOD or GPIO.
- A removable game cartridge containing non-volatile ROM that stores the game program.

Mechanical design requirements include:

- A 3D-printed console enclosure that houses the Basys-3 board.
- A cartridge slot that allows the game cartridge to be inserted and removed.

1.3 Constraints

The design of the gaming console is influenced by several system constraints related to hardware resources, performance requirements, and overall project scope. The system must operate within the available logic resources and Block RAM provided by the Basys-3 FPGA, which requires efficient use of hardware when implementing both the CPU and graphics

systems. In addition, generating a VGA video signal requires precise timing to maintain a stable 640 × 480 resolution at 60 Hz, meaning the system must balance graphics rendering with CPU execution without disrupting the display output. The project also involves integrating multiple subsystems, including digital logic, external peripherals such as the controller and cartridge interface, and mechanical components like the console enclosure. Finally, the development of the system is constrained by the timeline of a single semester, requiring the design, implementation, testing, and integration of all components to be completed within this time period.

Chapter 2: Background and Literature Review

2.1 Review of Existing Solutions

Early video game consoles used specialized hardware to generate graphics and execute game programs stored on external cartridges. Systems such as the Atari 2600 and Nintendo Entertainment System relied on simple processors combined with dedicated graphics hardware to produce video output and manage gameplay. These systems often used sprite-based graphics, where small graphical objects were stored in memory and rendered by hardware each frame.

Modern FPGA development boards make it possible to recreate similar architectures using programmable digital logic. Many FPGA-based projects implement custom processors, video generation hardware, and external memory interfaces that mimic the structure of early gaming systems. The system developed in this project follows a similar approach by combining a custom CPU, graphics subsystem, and cartridge-based memory system on an FPGA platform.

2.2 Theoretical Background

This project relies on several key concepts from digital system design. FPGAs allow digital circuits to be implemented using hardware description languages such as Verilog, enabling custom processors and hardware accelerators to be built directly in programmable logic.

The graphics subsystem generates a VGA video signal used to display graphics on a monitor. VGA output requires precise timing signals that control horizontal and vertical scanning of the display. For a resolution of 640 × 480 at 60 Hz, the system must generate synchronization signals while simultaneously producing pixel data. The graphics system also uses a sprite based rendering method where graphical objects are stored as small images and displayed based on position and control registers. The CPU interacts with the graphics hardware using a memory mapped interface, allowing software instructions to update graphical objects by writing values to specific memory locations.

2.3 Approach Justification

The architecture used in this project was selected to balance functionality with implementation complexity. The Basys-3 FPGA board provides sufficient programmable logic and built-in VGA output, making it well suited for implementing a custom gaming system. A custom CPU was designed to allow full control over the instruction set architecture and to demonstrate digital system design concepts. A sprite-based graphics architecture was chosen because it requires less memory than full frame-buffer graphics systems while still allowing multiple graphical objects to be displayed. Finally, the use of an external cartridge memory system allows game programs to be stored separately from the FPGA configuration, like the architecture used in early video game consoles.

Chapter 3: System Design and Architecture

3.1 System Overview

The retro gaming console is designed as a modular embedded system centered around the Basys-3 FPGA board. The system architecture replicates the structure of classic video game consoles, where a central processor executes game logic while dedicated graphics hardware generates video output. The overall system includes a custom CPU, GPU, system memory, an external game cartridge, and a player controller. The FPGA acts as the central component of the system and contains the CPU, GPU, memory modules, and VGA controller. External components include the display monitor, game cartridge, and game controller.

In the completed system, the CPU executes game instructions stored on the cartridge ROM while interacting with the graphics subsystem through a defined memory map. The graphics subsystem generates VGA output that is displayed on a monitor, while the controller provides real-time user input through the FPGA's I/O interfaces. Together, these subsystems operate as a unified gaming platform capable of executing gameplay logic, rendering sprite-based graphics, and processing player interaction in real time.

3.2 High-Level System Architecture

The system architecture divides the console into several interconnected components that allow the console to execute game programs and display graphics in real time. The FPGA console contains the CPU, GPU, and system memory, while external hardware includes the display monitor, cartridge ROM, and controller interface.

The architecture diagram shown in Figure 1 illustrates the relationship between these components. The CPU and GPU are implemented within the FPGA and communicate through

shared memory resources. The VGA controller generates a video signal that is sent to the display monitor at a resolution of 640 × 480 at 60 Hz. The cartridge subsystem provides external program memory using an EPROM device, allowing game programs to be loaded and executed by the system. Player input is provided through a custom controller connected to the FPGA using a PMOD interface.

The completed system successfully integrates the CPU, GPU, controller interface, and cartridge hardware into a unified gaming platform. During operation, the CPU executes game instructions stored on the cartridge ROM while interacting with the graphics subsystem through memory-mapped registers. The graphics subsystem renders sprite-based graphics and generates continuous VGA output, while controller inputs are processed in real time to update gameplay behavior. Final hardware testing confirmed stable communication between all major subsystems and successful execution of gameplay functionality on the FPGA platform.

3.3 Subsystem Breakdown

The gaming console architecture consists of several subsystems that together implement the functionality of the platform. These subsystems include hardware components implemented on the FPGA, supporting software tools used during development, and mechanical components designed for the final physical console.

3.3.1 Hardware Subsystems

CPU Architecture

The CPU is responsible for executing instructions that control game logic and overall system behavior. The architecture implements a Von Neumann memory organization, meaning that instructions and data share the same physical memory and

common address bus. The processor uses an 8-bit data path and a 16-bit unified address space, providing a balance between hardware simplicity and practical memory capacity. This allows sufficient addressable space for program storage, memory-mapped peripherals, and graphical data while keeping the ALU and register file compact.

The CPU supports R-type, I-type, memory, and J-type instructions. Its data path is divided into five main subsystems: instruction fetch, instruction decode and register access, effective address generation, execute, and register writeback. During instruction fetch, the 16-bit program counter provides the next memory address, and two sequential byte reads are combined to form one 16-bit instruction in the instruction register. Since memory is unified, access is time-multiplexed between instruction fetch and data operations through a shared interface.

The execution path includes a register file with an 8-bit dual-read, single-write structure, an 8-bit ALU for arithmetic and logic operations, and a dedicated address adder for effective address computation. The ISA uses a 4-bit opcode and supports arithmetic and logic instructions, immediate operations, memory access using base-plus-offset addressing, comparison, conditional branching, and absolute jumps. Final testing confirmed correct instruction execution, register updates, branching behavior, and memory access functionality within the completed processor architecture.

Graphics Subsystem

The graphics subsystem is responsible for generating the video output displayed on a monitor and rendering graphical objects within the game environment. The graphics system is implemented on the Basys-3 FPGA using Verilog and produces a VGA signal compatible with

standard displays. The VGA controller generates the synchronization signals and pixel timing required to support a resolution of 640 × 480 at 60 Hz.

The graphics architecture supports sprite-based rendering, allowing multiple graphical objects to be displayed and updated independently on the screen. These objects include the player character, enemy objects, and gameplay obstacles. Sprite attributes such as position, tile index, and enable flags are stored in registers that are updated by the processor during gameplay execution. Additional logic modules support functionality including object movement, gravity calculations, and collision detection between game entities.

Final system testing confirmed stable VGA output and successful rendering of sprite-based gameplay through the FPGA graphics pipeline. The graphics subsystem successfully interacted with the CPU through memory-mapped registers to update object positions and game states in real time.

Memory Architecture

The memory architecture of the system follows a unified Von Neumann model, where both instructions and data share the same physical memory space. The CPU uses a 16-bit address space and an 8-bit data path, allowing access to up to 64 KB of memory while keeping the processor hardware compact. This approach reduces overall hardware complexity compared to separate instruction and data memories while still providing sufficient memory resources for gameplay execution.

A 2-to-1 multiplexer selects the memory address source. One input comes from the program counter during instruction fetch, while the other comes from the calculated effective address during load and store operations. This shared addressing scheme allows the CPU to use a

single memory interface for both program execution and data access. FPGA Block RAM is used for temporary storage, graphical information, and system registers, while the external cartridge subsystem provides non-volatile program memory for game execution.

The processor also uses base-plus-offset addressing for memory operations. In this scheme, a register provides the base address and a signed immediate provides the displacement, allowing efficient access to arrays, structured data, and memory-mapped registers. Final testing verified correct operation of memory reads, writes, and address generation throughout the system.

Controller Hardware

The controller subsystem provides an interface between the player and the gaming system. A custom controller printed circuit board (PCB) was developed to allow player input through push buttons connected to the FPGA. The controller connects to the Basys-3 FPGA board through the PMOD interface, which provides general-purpose input/output connections.

The controller PCB includes tactile push buttons representing directional movement and action inputs used during gameplay. Each button is connected through pull-up resistors so that the FPGA receives a stable input signal when the button is not pressed. When a button is pressed, the corresponding signal changes state and is detected by the FPGA logic. Final testing confirmed reliable detection of player inputs during gameplay operation.

Cartridge Hardware

The cartridge subsystem provides external memory storage for the gaming system and allows game data to be loaded without modifying the FPGA configuration. The cartridge PCB interfaces with the FPGA and stores program data using a 27C512 EPROM. Because the

EPROM uses a parallel address interface, SN74HC573 octal latches are used to implement address latching. This allows the address bus to be multiplexed, reducing the number of FPGA input/output pins required to access the memory device.

The FPGA operates using 3.3 V logic levels, while the EPROM uses 5 V. To ensure safe communication between these devices, SN74LVC8T245 bidirectional level shifters translate voltage levels between the FPGA and the memory chip. Additional circuitry protects and stabilizes the power system. A 1 A polyfuse prevents excessive current draw from the FPGA board, and decoupling capacitors placed near integrated circuits reduce electrical noise and stabilize the power supply. The cartridge PCB also includes a boost converter and inductor used to regulate voltage for the memory device.

Final hardware validation confirmed successful communication between the FPGA and cartridge subsystem, allowing game programs stored on the EPROM to be accessed and executed by the console.

3.3.2 Software Components

Custom Assembler

The assembler was written in Python and translates readable assembly code into machine code executable by the custom processor implemented on the FPGA. The assembler reads an assembly source file containing instructions defined by the project's custom instruction set architecture. Each instruction is parsed and converted into its corresponding binary opcode based on the defined instruction format, including support for R-type, I-type, memory, and J-type instructions. The assembler then outputs the resulting machine code into a memory initialization file (program.mem) that is loaded into the FPGA program memory during synthesis.

This tool simplifies software development by allowing programs to be written using readable assembly instructions rather than raw binary values. The custom ISA uses a 4-bit opcode field and supports register-to-register ALU operations, immediate arithmetic and logic, memory access, comparison, conditional branches, and absolute jumps.

The assembler was successfully used to create and execute test programs for gameplay functionality, CPU verification, and graphics control. Final testing confirmed correct opcode generation, memory initialization, and compatibility with the implemented processor architecture.

3.4 Component Selection Analysis

The selection of system components was based on compatibility with the FPGA platform, ease of integration, and the ability to support the functional requirements of the gaming console. For external program storage, a 27C512 EPROM was selected as the cartridge memory device. This device provides sufficient capacity to store game data and graphical assets while supporting a parallel interface that can be accessed by the FPGA. Address latching for the cartridge memory was implemented using SN74HC573 octal latches. These components allow the address bus to be multiplexed, which reduces the number of FPGA I/O pins required for the memory interface. Because the FPGA operates at 3.3 V logic levels while the EPROM uses 5V, SN74LVC8T245 bidirectional level shifters were selected to safely translate voltage levels between the two devices. This ensures reliable communication while protecting the FPGA from higher voltage levels. Additional supporting components such as polyfuses, decoupling capacitors, and voltage regulation circuitry were included to ensure stable and safe operation of the hardware system.

3.5 Budget and Bill of Materials

The overall project budget includes estimated labor costs, physical materials, and supporting resources required to develop the retro gaming console. Because the project is completed as part of an academic laboratory course, most of the core hardware such as the Basys-3 FPGA development board was provided by the stockroom. As a result, most of the project cost is associated with labor and materials.

The estimated labor cost assumes a team of three students working approximately 15 hours per week over the duration of the semester. An hourly rate of $15 per hour was used to estimate engineering labor costs for the design, implementation, and testing of the system. Additional labor categories such as instructor supervision and lab assistance were also included to represent the full development environment. According to the project budget estimate, the total projected labor cost for the system is approximately $18,000. Material costs for the system are relatively small compared to the labor cost. Physical components include printed circuit boards for the controller and cartridge hardware, integrated circuits such as the EEPROM and address latches used in the cartridge interface, passive electronic components, and 3D printing filament used to fabricate the console enclosure and cartridge housing.

When including materials, labor, and estimated overhead, the total projected project cost is approximately $30,000 according to the budget model. However, the direct material cost required to physically build the console hardware is significantly lower due to the use of existing laboratory equipment and provided FPGA hardware.

Chapter 4: Implementation Details

4.1.1 CPU Hardware Implementation

The processor architecture includes several core components such as the program counter, register file, arithmetic logic unit (ALU), and instruction decoding logic. The data path architecture defines how instructions flow through the processor and how data moves between registers, the ALU, and memory. The ALU performs arithmetic and logical operations required for program execution, while the control logic determines how each instruction is executed.

The instruction set architecture (ISA) defines the structure of instructions supported by the processor. Each instruction contains fields specifying the operation type, registers used, and immediate values when required. The instruction format was designed to support arithmetic operations, memory access, and branching instructions. Figure 2 shows the CPU data path architecture used in the final implementation.

Final validation of the processor confirmed correct instruction execution, arithmetic functionality, memory access operations, and branching behavior during simulation and hardware testing. The CPU successfully interacted with memory-mapped graphics registers and executed assembly programs used for gameplay functionality.

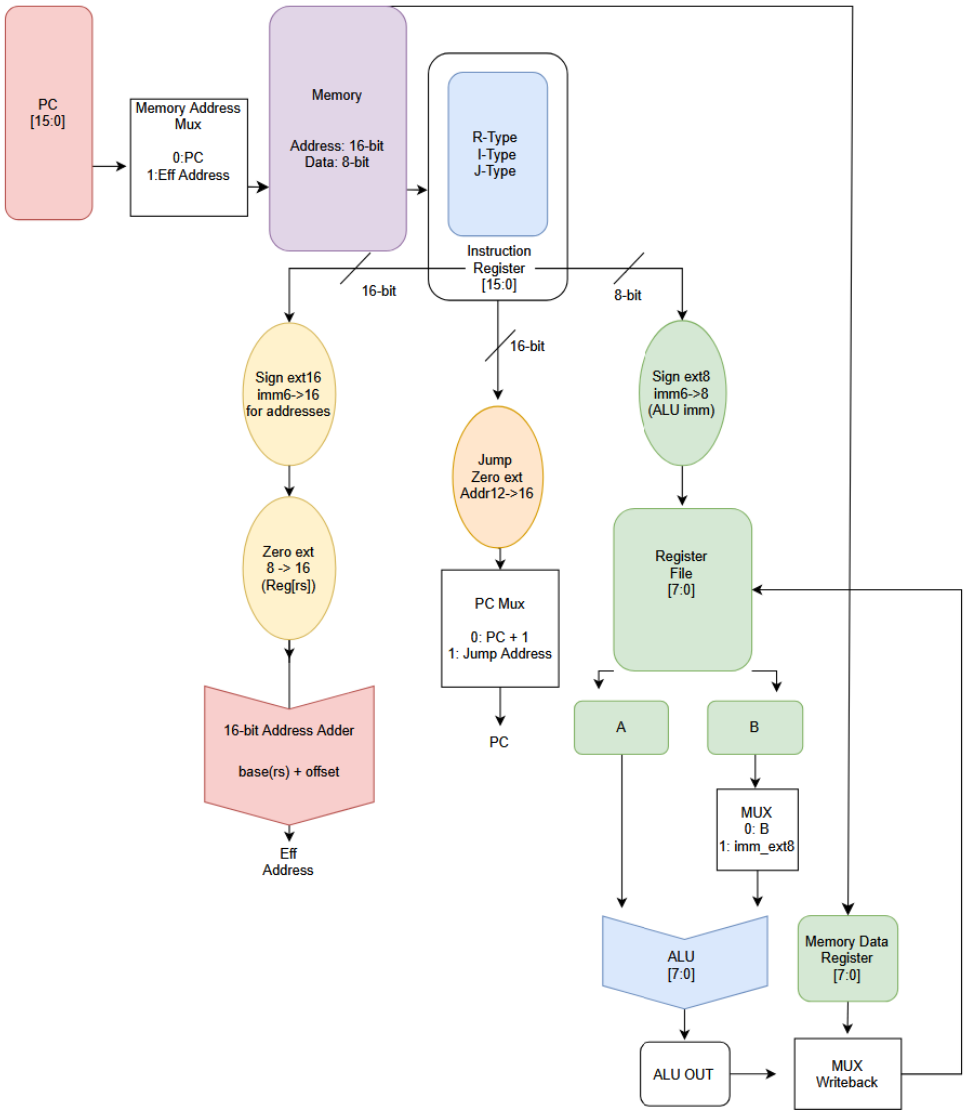

Figure 2. CPU Datapath Architecture

4.1.2 Graphics Hardware Implementation

The graphics subsystem is responsible for generating the VGA output signal used to display graphics on a monitor. The VGA controller produces synchronization signals and pixel

data required for a standard VGA display operating at 640 × 480 resolution with a refresh rate of 60 Hz.

The graphics architecture includes sprite registers that store information about graphical objects displayed on the screen. These registers store parameters such as sprite position, tile index, and enable flags. The CPU updates these registers during gameplay execution to control what appears on the display.

Additional graphics logic handles sprite rendering, object movement, collision detection, and screen updates during gameplay operation. The completed graphics subsystem successfully generated stable VGA output and rendered sprite-based gameplay on an external display. Figure 3 shows the final graphics subsystem rendering gameplay through the FPGA VGA interface.

Figure 3. Final gameplay rendered through the FPGA graphics subsystem

4.1.3 Controller Hardware Design

A custom controller PCB was designed to provide user input for the gaming system. The controller connects to the FPGA using the PMOD interface available on the Basys-3 board. The

controller uses push buttons to represent directional inputs and action commands used during gameplay. Each button is connected through pull-up resistors shown in Figure 4, to ensure that the FPGA receives stable input signals when the buttons are not pressed. When a button is pressed, the corresponding signal line changes state and is detected by the FPGA. This simple interface allows the FPGA to read player input through its general-purpose input pins while minimizing the complexity of the controller circuitry. The actual PCB is shown below in Figure 5.

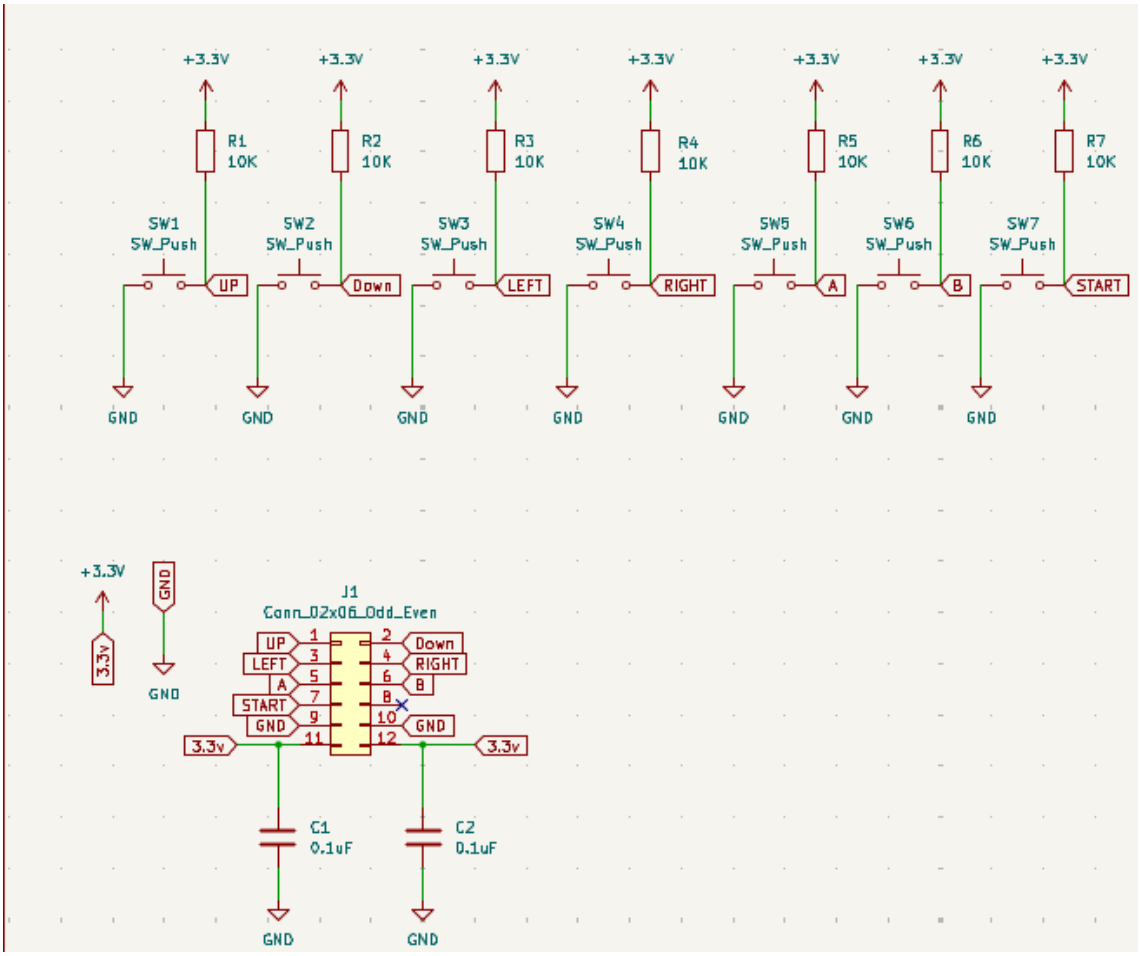

Figure 4. Controller PCB Schematic

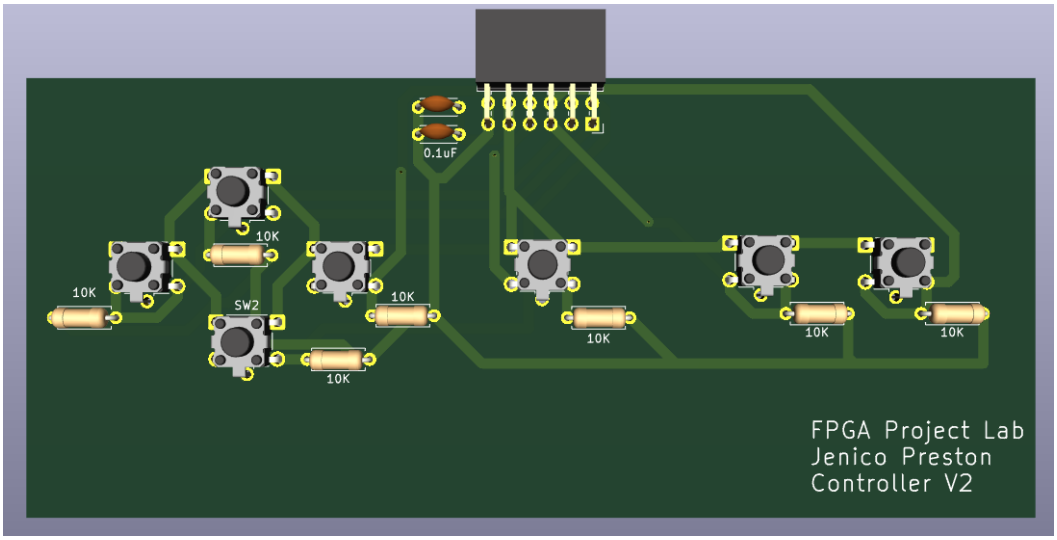

Figure 5. Controller PCB 3D Model

4.1.4 Cartridge Hardware Design

The cartridge subsystem was designed to provide external memory storage for game data and graphical assets. The cartridge PCB uses a 27C512 EPROM to store program data outside of the FPGA. This approach allows game data to be stored externally while the FPGA reads instructions and graphics information through the cartridge interface. Because the EPROM requires a parallel address interface, SN74HC573 octal latches are used to implement address latching. These latches allow the address bus to be multiplexed so that fewer FPGA input/output pins are required to access the memory device. The EPROM operates at 5 V logic levels, while the FPGA uses 3.3 V logic. To ensure safe communication between these devices, SN74LVC8T245 bidirectional level shifters are used to translate voltage levels between the FPGA and the memory device. A 1A polyfuse is used to prevent excessive current draw from the FPGA board, while decoupling capacitors placed near integrated circuits help stabilize the power supply and reduce electrical noise. The cartridge PCB also includes a boost converter circuit and

inductor to ensure proper voltage regulation for the EEPROM. The schematic for the cartridge PCB is shown below in Figures 6-9.

4.2.1 Custom Assembler

A custom assembler was developed using Python to translate assembly language programs into machine code compatible with the custom CPU architecture. The assembler reads assembly instructions, parses the instruction fields, and generates binary machine code used by the processor. The output of the assembler is a memory initialization file (program.mem) that can be loaded into the FPGA's program memory. Figure 10 shows an example assembly program and the generated machine code used to test the CPU.

Figure 10. Demonstration of the assembler functioning.

4.2.2 CPU Test Programs

Test programs were created to verify that the CPU correctly executes instructions and updates memory values during gameplay operation. One example program writes data to sprite registers that control graphical objects displayed on the screen. The test sequence shown in Figure 10 demonstrates how the CPU updates sprite attributes using a sequence of instructions.

This sequence writes values to registers corresponding to sprite position, tile index, and sprite enable flags.

Additional test programs were developed to validate arithmetic operations, branching instructions, memory access operations, and interactions between the CPU and graphics subsystem. These programs were used during simulation and hardware testing to confirm correct instruction execution, register updates, memory writes, and communication with memory-mapped graphics registers. Final validation confirmed that the processor successfully executed assembled game programs and correctly interacted with the graphics subsystem during gameplay operation.

4.3 Final System Integration

The final implementation successfully integrated the CPU, graphics subsystem, controller hardware, cartridge interface, and custom assembler into a complete FPGA-based gaming platform. The CPU executed game programs stored on the external cartridge ROM while communicating with the graphics subsystem through memory-mapped registers. The graphics subsystem rendered sprite-based gameplay and generated stable VGA output at 640 × 480 resolution with a refresh rate of 60 Hz.

The controller hardware successfully provided real-time player input through the PMOD interface, allowing gameplay interaction through the custom PCB controller. The cartridge subsystem successfully interfaced with the FPGA and allowed externally stored game programs to be accessed and executed by the system.

Final system testing demonstrated stable operation of all major subsystems operating simultaneously on the Basys-3 FPGA platform. The completed system successfully executed

gameplay functionality, rendered graphics to an external VGA display, processed controller input, and loaded program data from cartridge memory, demonstrating the successful implementation of a fully functional retro-style FPGA gaming console.

Chapter 5: Validation and Testing Results

5.1 Test Plan

The goal of the testing process was to verify that the major subsystems of the project operate as expected before full system integration. Because the console architecture is composed of multiple independent modules, testing was performed at the subsystem level using simulation and hardware demonstrations. The primary components tested include the CPU architecture, graphics subsystem, assembler functionality, and hardware interface designs. Verilog simulations were used to verify instruction execution, data path behavior, and register updates within the CPU. Additional test programs were written to validate communication between the processor and graphics registers. Graphics functionality was tested through a GPU demonstration that confirmed the FPGA could generate VGA output and render graphical objects on the screen. Hardware designs for the controller and cartridge subsystems were verified through schematic review and PCB layout validation to ensure proper signal routing and electrical compatibility. This testing approach allowed each major subsystem to be validated independently before integration of the full console architecture.

5.2 Simulation Results

CPU Instruction Execution

Simulation results demonstrate that the processor correctly executes instructions defined in the custom instruction set architecture. The CPU supports R-type, I-type, memory, and J-type

instructions, and the simulations verified correct instruction decoding, ALU control, register access, and writeback behavior. The unified memory interface was also validated by showing that instruction fetch and data operations correctly share the same memory system through time-multiplexed access.

Example test programs were used to verify arithmetic operations, immediate instructions, register updates, and program flow control. Additional tests confirmed effective address generation for memory instructions using base-plus-offset addressing. These simulations showed that the data path correctly processes instructions, updates registers, and routes memory addresses as expected.

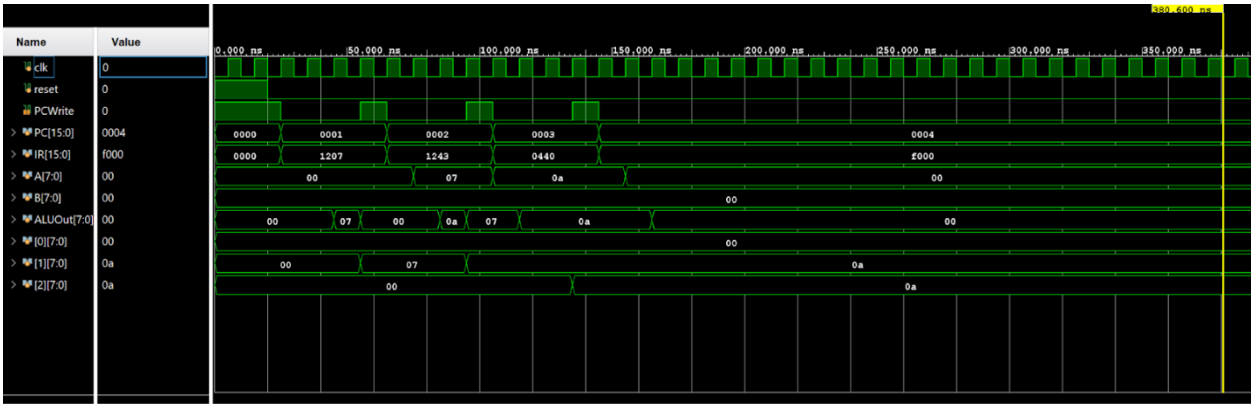

Figure 11. Simulation waveform of the working CPU ISA

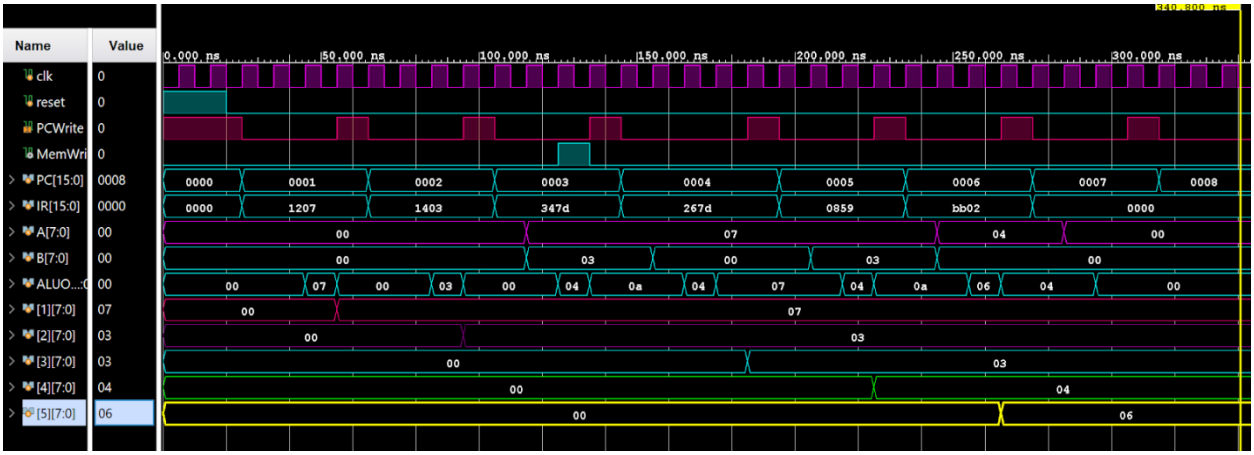

Figure 12. Another simulation test of the working CPU ISA

Sprite Register Test

A specific test program was created to verify that the CPU could correctly write values to sprite registers used by the graphics subsystem. The program updates register responsible for sprite position, tile index, and sprite enable flags. The instruction sequence used in the test program included operations such as ADDI, STORE, and JMP to modify sprite parameters in memory. This test demonstrated that the CPU can successfully update memory mapped registers that control graphical objects shown in Figure 13.

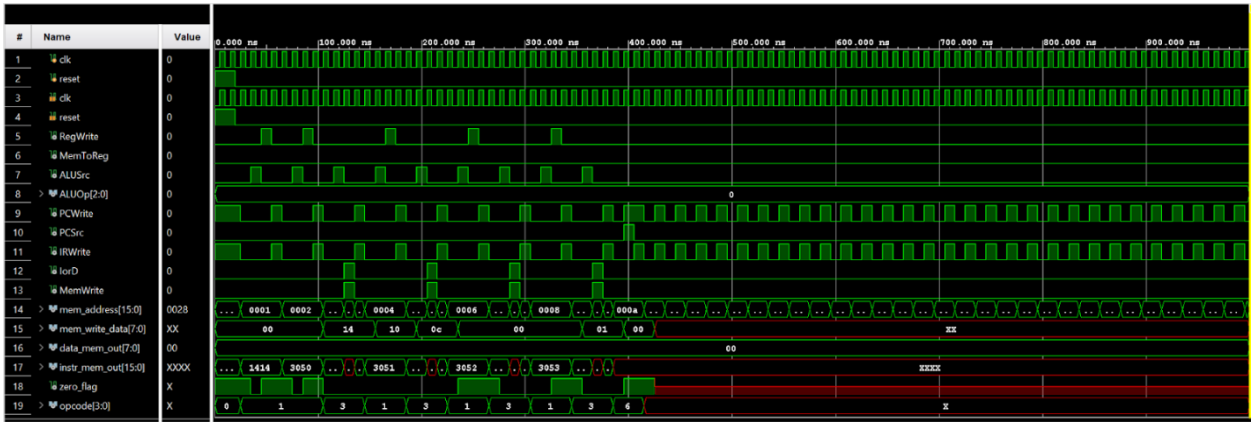

Figure 13. Waveform of working CPU Sprite Register Test

5.3 Resource Utilization Results

Synthesis reports generated during FPGA implementation were used to evaluate resource usage on the Basys-3 board. The final implementation uses the following FPGA resources shown in Figure 14. These results indicate that the design uses only a portion of the available FPGA resources, leaving sufficient capacity for additional modules such as full system integration and expanded game logic.

LUT	FF	BRAM
2626	716	36
2568	716	36

Figure 14. Design Report showing BRAM Tile, LUT, and Flip Flop usage

5.4 Analysis and Discussion

The testing process demonstrated that the major components of the system operated correctly both independently and as an integrated platform. Simulation and hardware testing confirmed that the CPU architecture correctly executed instructions and updated registers and memory values during gameplay operation. The results also validated the unified Von Neumann memory design, where instruction fetch and data operations share the same memory interface. Although this organization simplifies the hardware implementation, it also requires memory accesses to be serialized, representing an important design tradeoff between simplicity and performance. The processor's 8-bit data path and 16-bit address space were selected to balance hardware simplicity with memory flexibility. Testing confirmed that this combination supported

practical program execution while keeping the architecture compact enough for efficient FPGA implementation. In addition, the use of base-plus-offset addressing improved memory access flexibility and allowed efficient interaction with memory-mapped graphics registers and structured data.

The sprite register test further verified that the CPU successfully interacted with memory-mapped graphics hardware. Final graphics testing confirmed that the FPGA generated stable VGA output at 640 × 480 resolution with a refresh rate of 60 Hz while rendering sprite-based gameplay through the implemented graphics pipeline. The CPU, graphics subsystem, controller hardware, and cartridge interface were successfully integrated through the shared memory architecture and operated simultaneously during gameplay execution.

Final system testing also confirmed successful communication between the FPGA and external hardware subsystems. The controller hardware reliably detected player input through the PMOD interface, while the cartridge subsystem successfully loaded and executed externally stored game programs using the EPROM-based memory interface. These results demonstrate the successful implementation of a fully functional FPGA-based retro gaming console integrating custom digital hardware, external peripherals, and software development tools into a unified gaming platform

Chapter 6: Conclusion

6.1 Summary of Achievements

The goal of this project was to design and implement a fully functional retro-style gaming platform using an FPGA-based architecture. The completed system integrates a custom central processing unit, sprite-based graphics subsystem, external cartridge memory, and custom

controller hardware into a single gaming platform implemented on the Basys-3 FPGA board. The architecture replicates the structure of early video game consoles by combining programmable logic, graphics hardware, memory systems, and external peripherals into a unified system capable of executing and displaying a playable game. Several major components of the system were successfully implemented and validated through simulation and hardware testing. The graphics subsystem capable of rendering sprites and generating VGA video output at 640 × 480 resolution and 60 Hz was successfully implemented on the FPGA platform. The custom processor architecture correctly executed instructions, updated memory and graphics registers, and interacted with the graphics subsystem through memory-mapped communication. A custom assembler was also developed to translate assembly programs into machine code compatible with the processor architecture and was successfully used to execute gameplay programs on the system.

In addition to the FPGA logic implementation, supporting hardware subsystems were successfully developed and integrated into the final platform. A cartridge printed circuit board was implemented to provide external memory storage using a 27C512 EPROM device, allowing game programs to be loaded and executed by the console. A custom controller PCB was also implemented to provide real-time player input through the FPGA PMOD interface. Final system testing demonstrated stable operation of the CPU, graphics subsystem, controller hardware, and cartridge interface operating simultaneously on the FPGA platform. The completed system successfully executed game programs, rendered sprite-based graphics through VGA output, processed controller input, and accessed externally stored cartridge memory. These results demonstrate the successful implementation of a complete FPGA-based retro gaming console using custom digital hardware and supporting software tools.

6.2 Lessons Learned

Throughout the development of the project, several important engineering lessons were learned. One of the most significant challenges was coordinating the design of multiple subsystems simultaneously. The project required knowledge of digital logic design, hardware description languages, printed circuit board design, and embedded system architecture. Integrating these disciplines into a single project required careful planning and communication among team members. Another key lesson involved the importance of simulation and incremental testing when developing FPGA-based systems. Verifying individual modules through simulation helped identify potential design issues early and prevented larger integration problems later in development.

The project also highlighted the complexity of hardware-software interaction in embedded systems. Designing a custom instruction set and assembler required careful consideration of how the processor would interact with memory and graphics hardware. Overall, the project has provided a valuable experience in systems engineering and demonstrated the challenges involved in building a complete hardware platform from the ground up.

Appendix

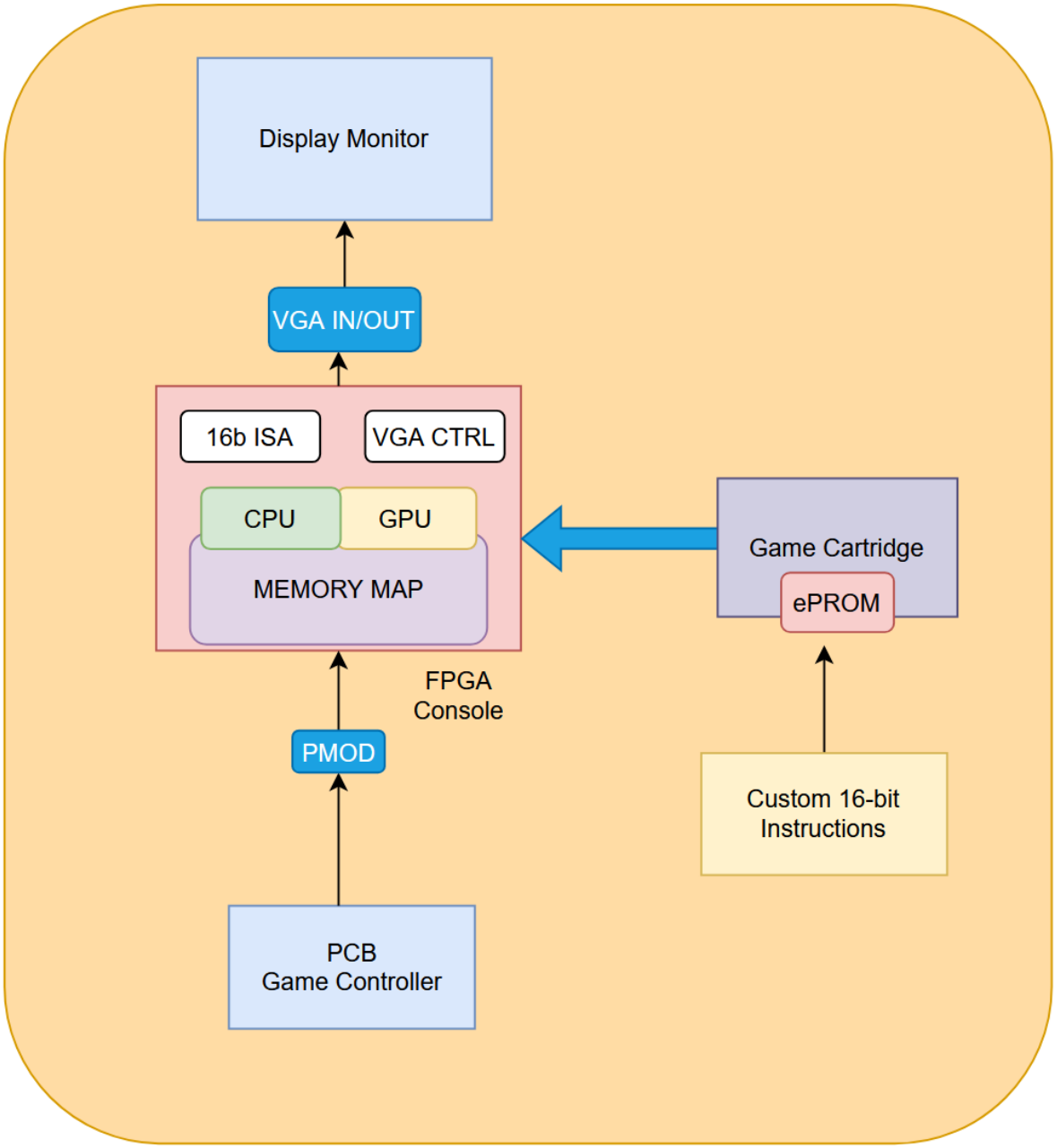

Figure 1. High-Level Datapath of the CPU Architecture implemented on the Basys 3 Board

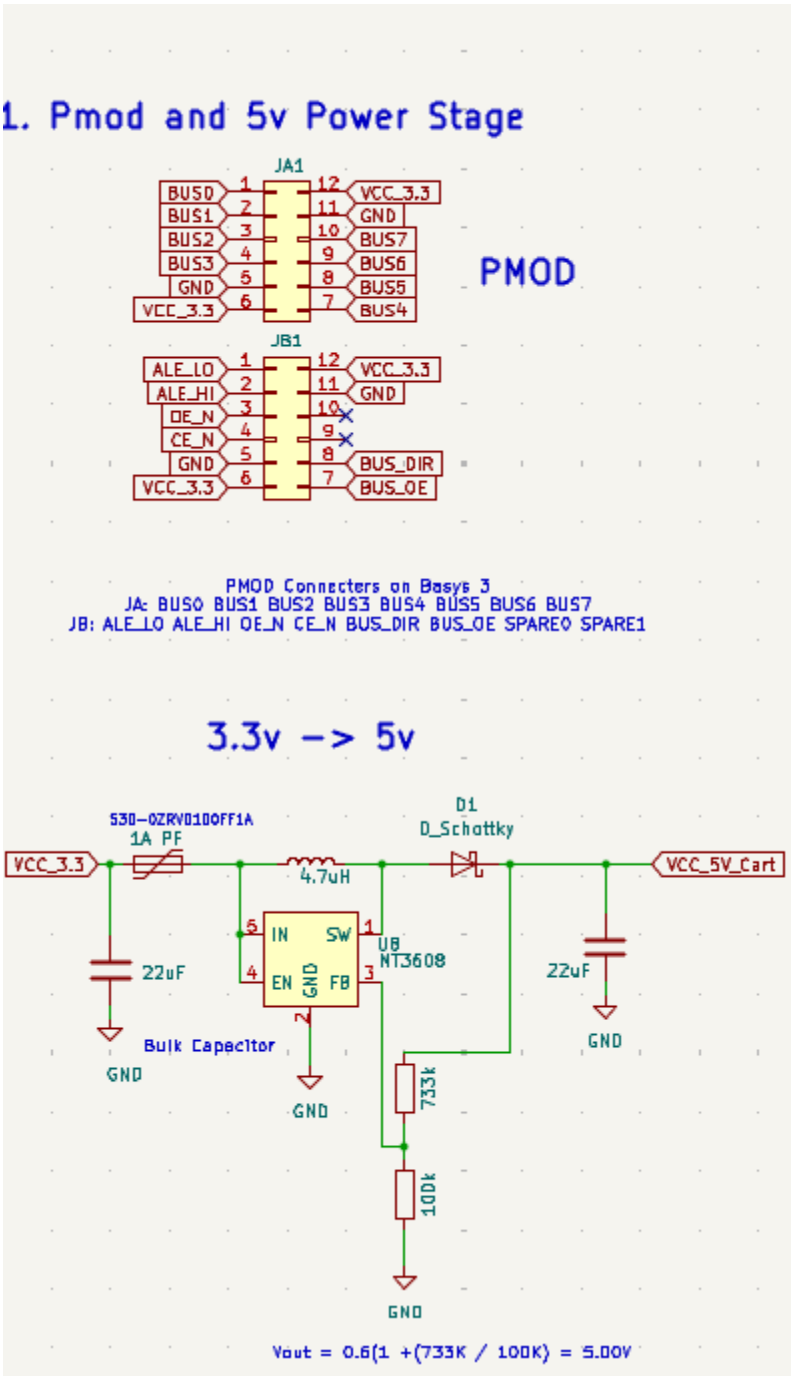

Figure 6. PMOD and Power Stage for cartridge PCB

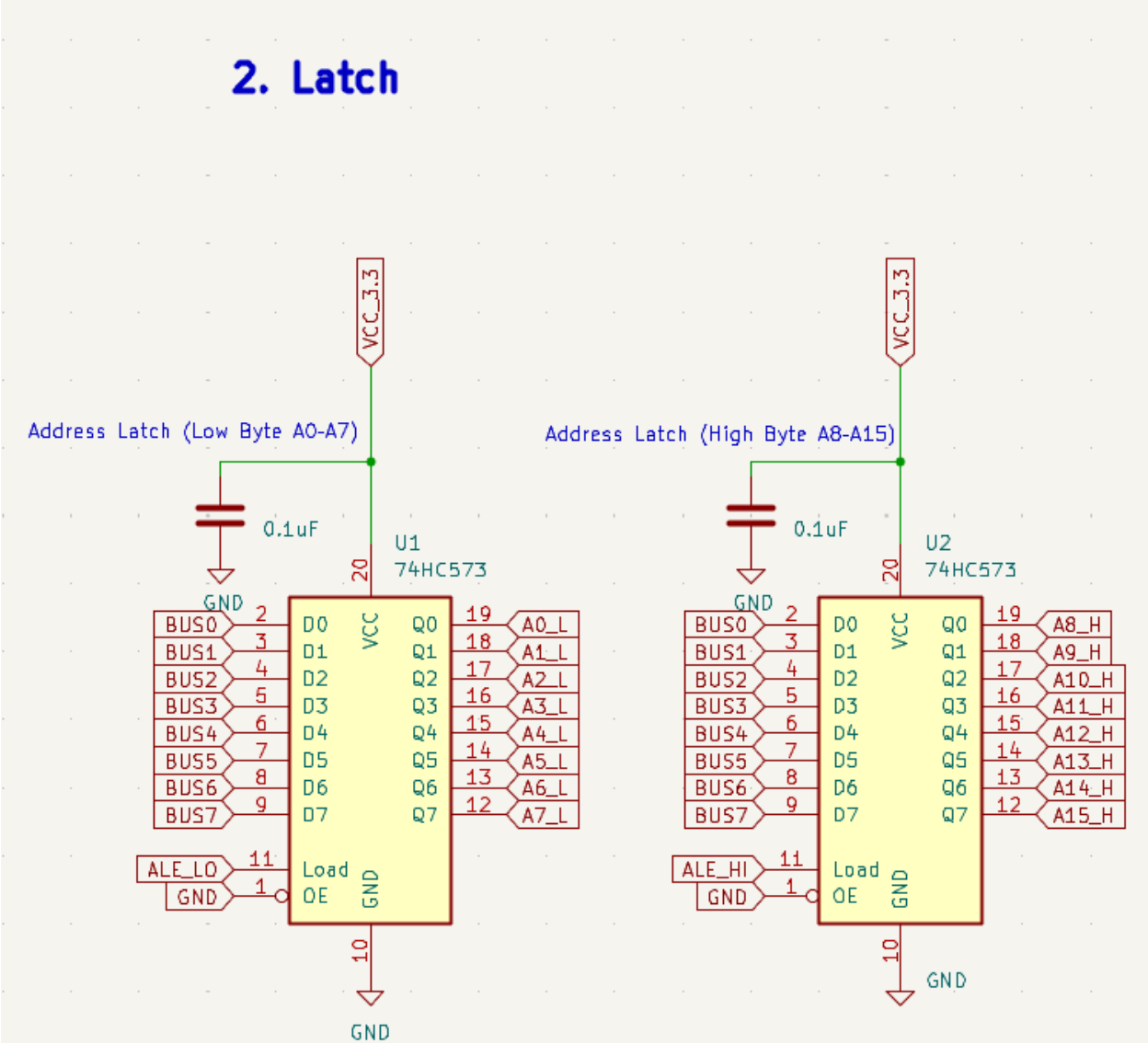

Figure 7. Latch for cartridge PCB

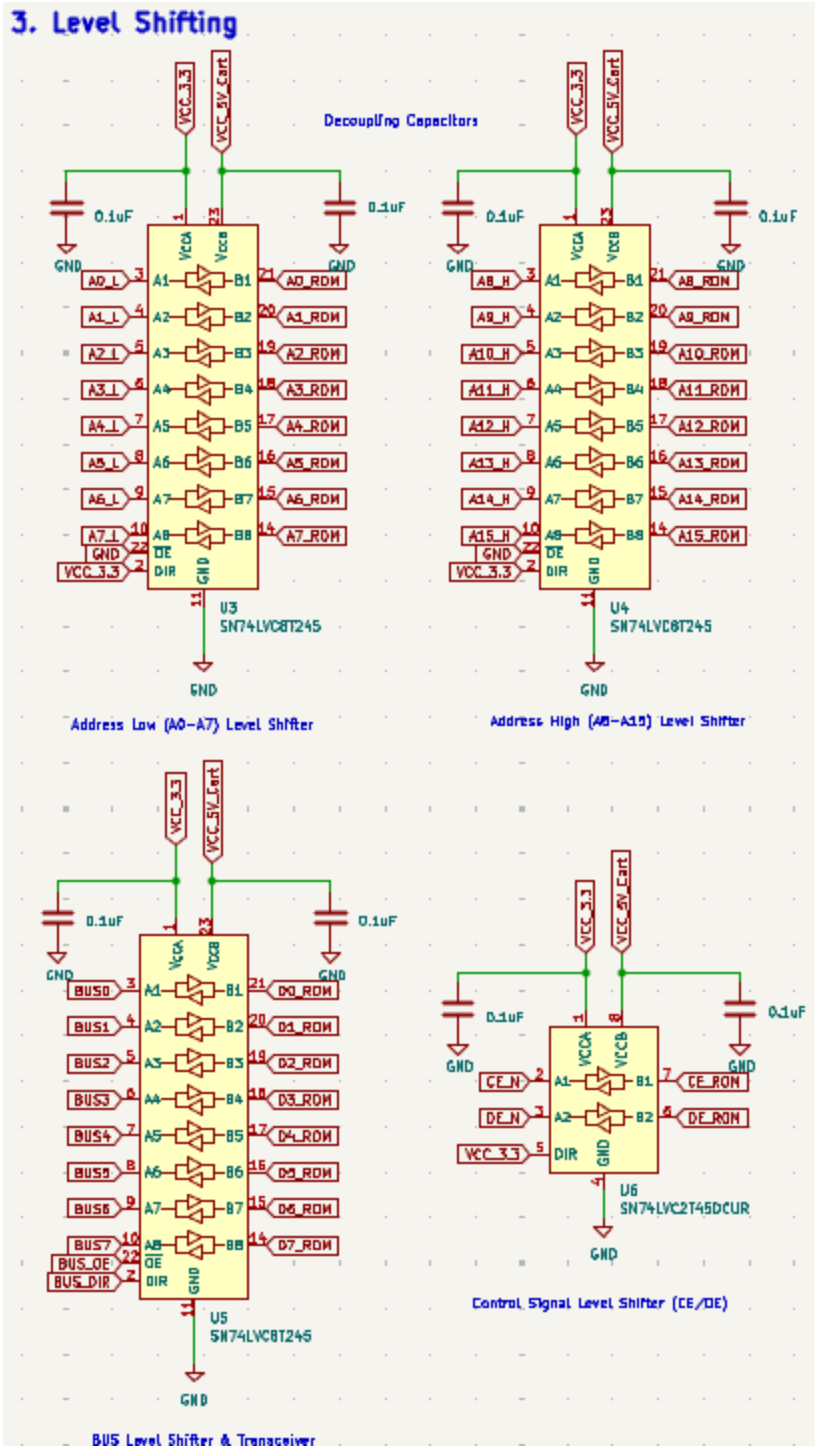

Figure 8. Level Shifters for cartridge PCB

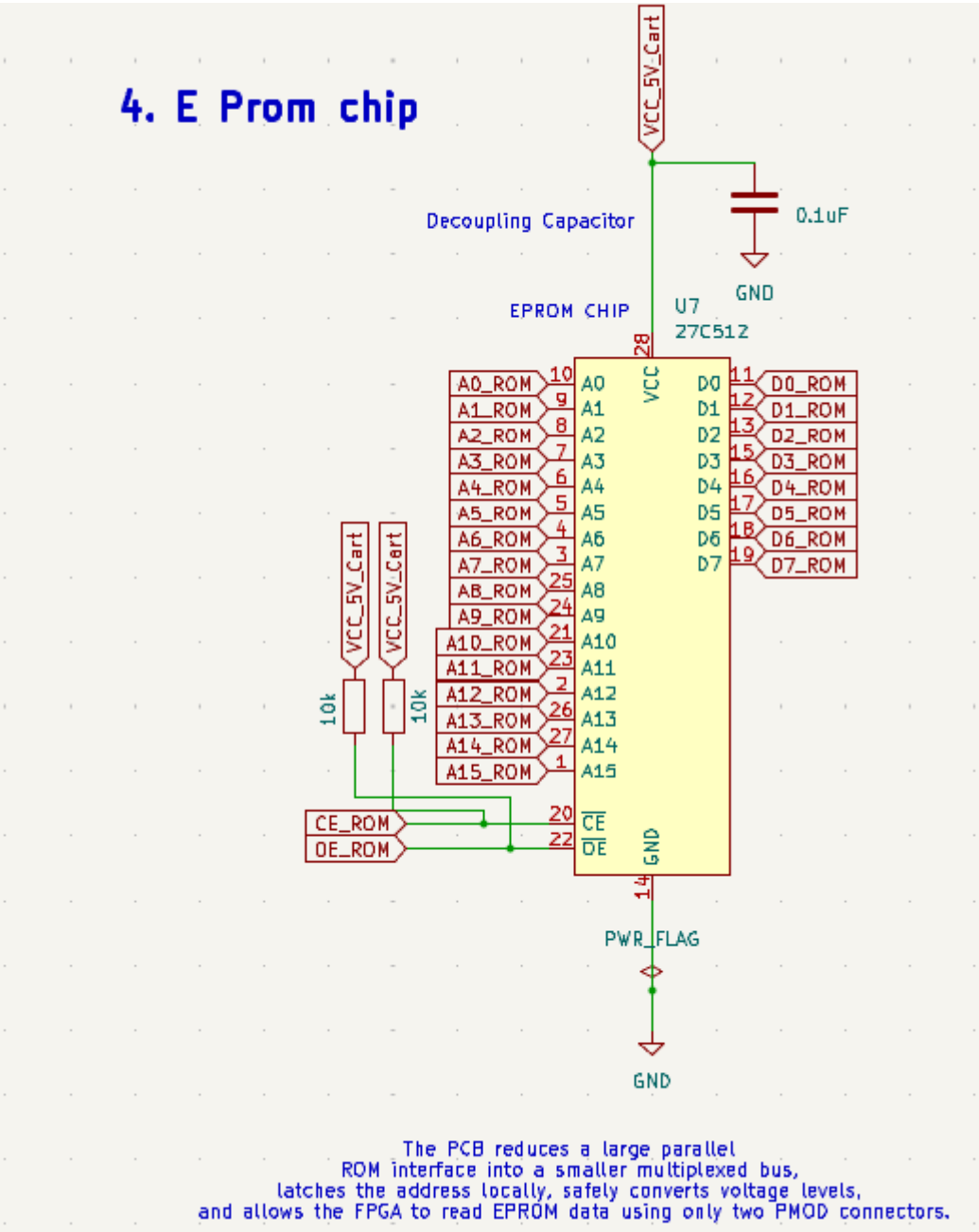

Figure 9. EPROM chip for cartridge PCB

FPGA Project Lab	Final Total			Total Estimate			Start Date	1/15/2026
Direct Labor:	Rate/Hr	Hrs		Rate/Hr	Hrs		**Today**	5/9/2026
Jose Gonzales	$15	148	$2,220	15	200	$3,000		
Tyler Wolfe	$15	144	$2,160	15	200	$3,000	**End Date**	5/2/2026
Jenico Preston	$15	146	$2,190	15	200	$3,000		
DL Subtotal(DL)		Subtotal:	$6,570.00		Subtotal:	$9,000		
Labor Overhead	Rate:	100%	$6,570.00	Rate:	100%	$9,000		
Total Direct Labor(TDL)			$13,140.00			$18,000		
Contract Labor:								
Lab Teaching Assistant	$40	0	$0	$40	10	$400		
Instructor	$200	0	$0	$200	6	$1,200		
Total Contract Labor(TCL)			$0			$1,600		
Direct Material Costs:								
3D Printer Filament	Cost:	20$/Spool	$60	Cost:	20$/Spool	$60		
EPROM Programmer			$79.95			$0		
Controller PCB & Components			$18			$20		
Cartridge PCB & Components			$57			$60		
PMOD Cables		3.99 each	$8.00			$8		
Total DMC			$223			$140		
Total TDL+TCL+TDM+TRM			$13,362.72			$19,740		
Business Overhead		55%	$7,349.50		55%	$10,857		
Total Cost		**Current**	$20,712.22		**Estimate**	$30,597		
SUMMARY								
Labor + OH	$13,140.00			$18,000				
Contract Labor	$0.00			$1,600				
Materials & Equip Rental	$223			$0.00				
Overhead	$20,712.22			$19,740	**% Budget Used**	86.61651246		
TOTAL	$34,074.94			$39,340	**Remaining**	$5,265		

Figure 15. Budget Report and Bill of Materials

Figure 16. Final Implementation of Hardware in 3D Printed Enclosures